

# Auction Portal — Testing Guide

## For the person reviewing this system

Version 0.1 (Phase 1) · September 2026

### 1. What this is

A website that collects bank auction listings from public sources and puts them in one searchable place. Banks sell seized property and vehicles when borrowers default; the notices are published, but scattered across dozens of websites in dozens of formats. This portal gathers them.

#### What it can do today

- Collects listings automatically from **BAANKNET**, the Government of India auction platform used by all 12 public sector banks
- Collects **repossessed vehicles** from the same platform
- Lets anyone search and filter by bank, state, city, property type, possession status and price
- Shows reserve price, EMD, auction dates and inspection windows
- Links every listing back to the bank's original notice
- Updates itself on a schedule with no one touching it

#### What it cannot do yet

This is Phase 1. The following are **deliberately** not built:

| Not yet                                    | Why                                     | When    |
|--------------------------------------------|-----------------------------------------|---------|
| ICICI, HDFC, Axis and other private banks  | Their notices are PDFs and need OCR     | Phase 2 |
| Auction platforms (C1 India, AuctionTiger) | Mixed HTML and PDF formats              | Phase 3 |
| Newspaper notices                          | Hardest source; needs vernacular OCR    | Phase 5 |
| Email or WhatsApp alerts                   | Needs user accounts first               | Phase 4 |
| Bidding or payments                        | <b>Never.</b> We are not an auctioneer. | —       |

Please do not report these as defects. They are scheduled work.

### 2. What you are testing against

At the time of writing the system holds:

|             |                                                 |
|-------------|-------------------------------------------------|
| Listings    | <b>85</b> (84 published, 1 withheld on purpose) |
| Properties  | 60                                              |
| Vehicles    | 25                                              |
| Banks       | 11 public sector banks                          |
| Cities      | 64                                              |
| Price range | ₹40,375 to ₹44.65 crore                         |

This is a **sample**, not full coverage. BAANKNET publishes around 72,000 properties; we deliberately crawl slowly and politely, so building the full set takes time. The point of Phase 1 is to prove the machinery works.

### 3. Starting the system

You need two things running: the **database** and the **portal**.

Open PowerShell in the project folder ( `D:\Anubhav Jain bankproject` ) and run these in order.

#### Step 1 — start the database

```
.\.venv\Scripts\python.exe scripts\setup_postgres.py --start
```

Expected:

```
binaries : present
data dir : initialised
server   : running (port 5433)
```

The database does **not** start automatically when Windows boots. If you restart the machine, run this again.

#### Step 2 — start the portal

```
.\.venv\Scripts\python.exe -m auction_portal serve
```

Expected:

```
portal running at http://127.0.0.1:8000
api docs at      http://127.0.0.1:8000/api/docs
press Ctrl+C to stop
```

Leave this window open. The portal runs until you press **Ctrl+C**.

#### Step 3 — open it

Go to **<http://127.0.0.1:8000>** in any browser.

## 4. Test checklist

Work through these in order. Each has a clear expected result.

### A. The home page

| #  | Do this                                 | Expect                                                                 |
|----|-----------------------------------------|------------------------------------------------------------------------|
| A1 | Open <code>http://127.0.0.1:8000</code> | A list of properties, newest auction first                             |
| A2 | Look at the top bar                     | Counts for listings, states and banks                                  |
| A3 | Check a price                           | Shown as <b>Rs 37.65 L</b> or <b>Rs 3.38 Cr</b> , never <b>3765000</b> |
| A4 | Check an auction date                   | Indian format and IST, e.g. <b>29 Sep 2026, 11:00</b>                  |
| A5 | Scroll to the bottom                    | A disclaimer, and a statement that we are not a bank                   |

### B. Searching

| #  | Do this                                            | Expect                                             |
|----|----------------------------------------------------|----------------------------------------------------|
| B1 | Type <b>Kanpur</b> in the search box, press Search | Only Kanpur listings                               |
| B2 | Search <b>Baroda</b>                               | Listings from Bank of Baroda                       |
| B3 | Search <b>Antarctica</b>                           | "Nothing matches those filters", not an error page |
| B4 | Clear the search                                   | All listings return                                |

### C. Filtering

| #  | Do this                                                     | Expect                                              |
|----|-------------------------------------------------------------|-----------------------------------------------------|
| C1 | Click <b>Gujarat</b> in the State list                      | Only Gujarat listings; a "Gujarat ×" chip appears   |
| C2 | Look at the State list again                                | Other states <b>still visible</b> with their counts |
| C3 | Add a property type filter                                  | Both filters apply together                         |
| C4 | Click the × on a chip                                       | That filter is removed, the other stays             |
| C5 | Click <b>Clear all</b>                                      | Back to everything                                  |
| C6 | Enter Min <b>1000000</b> , Max <b>5000000</b> , click Apply | Only listings in that price band                    |

C2 is worth checking carefully. Selecting Gujarat must not reduce the state list to only Gujarat, or you could never switch state without clearing the filter first.

## D. Sorting and paging

| #  | Do this                                         | Expect                               |
|----|-------------------------------------------------|--------------------------------------|
| D1 | Sort by <b>Price (lowest first)</b>             | Cheapest at the top                  |
| D2 | Sort by <b>Price (highest first)</b>            | The ₹44.65 crore listing at the top  |
| D3 | Sort by <b>Auction date (soonest)</b>           | Nearest auction first                |
| D4 | Click <b>Next</b> at the bottom                 | Page 2; "Page 2 of N" shown          |
| D5 | Filter by a state, then look at the page number | Resets to page 1, not a blank page 7 |

## E. A single listing

| #  | Do this                                          | Expect                                                     |
|----|--------------------------------------------------|------------------------------------------------------------|
| E1 | Click any listing                                | A detail page opens                                        |
| E2 | Look for the money                               | Reserve price, EMD, bid increment, auction opening time    |
| E3 | Find <b>"View the original notice"</b>           | Opens the bank's own page on baanknet.com                  |
| E4 | Look under that button                           | "first seen" and "last verified" times                     |
| E5 | Open a listing tagged <b>symbolic possession</b> | An orange warning that occupants may still be living there |
| E6 | Open one tagged <b>DRT</b>                       | A warning that it is not a standard SARFAESI auction       |
| E7 | Click a city in the breadcrumb                   | Filters to that city                                       |

## F. Things that must NOT happen

These are the important ones.

| #  | Check                                                                                         | Expect                                                 |
|----|-----------------------------------------------------------------------------------------------|--------------------------------------------------------|
| F1 | Search any page for the word "borrower"                                                       | <b>Never appears.</b> We do not publish who defaulted. |
| F2 | Search for "guarantor"                                                                        | <b>Never appears</b>                                   |
| F3 | Visit <a href="http://127.0.0.1:8000/listing/999999">http://127.0.0.1:8000/listing/999999</a> | A clean "Listing not found" page, not a crash          |
| F4 | Look for a "Bid now" or "Pay EMD" button                                                      | <b>Must not exist.</b> We are not an auctioneer.       |
| F5 | Look for a map pin on every listing                                                           | Some have none — that is <b>correct</b> , see below    |

**F5 explained.** Some listings show no location. The source data contains wrong coordinates — one Azamgarh property was published in the Arabian Sea. Where we cannot verify a coordinate falls inside the state it claims, we show nothing. A wrong pin on a ₹38 lakh property is worse than no pin.

## G. The JSON API

For anyone technical on your side.

| #  | Do this                                              | Expect                                    |
|----|------------------------------------------------------|-------------------------------------------|
| G1 | Open <code>http://127.0.0.1:8000/api/docs</code>     | Interactive API documentation             |
| G2 | Open <code>http://127.0.0.1:8000/api/listings</code> | JSON with a <code>disclaimer</code> field |
| G3 | Open <code>http://127.0.0.1:8000/api/health</code>   | <code>{"status": "ok", ...}</code>        |
| G4 | Search the JSON for "borrower"                       | Not present                               |

## 5. Checking the automatic side

The portal is only half the system. The other half collects the data. Open a **second** PowerShell window, leaving the portal running.

### Is everything healthy?

```
.\.venv\Scripts\python.exe -m auction_portal health
```

Expected:

```
[ok ] baanknet          healthy    last run 27 Sep 00:49
[ok ] baanknet_vehicle healthy    last run 27 Sep 00:45
```

**FAIL** means a source has stopped producing data — usually because the website changed and the collector needs updating.

### Collect more listings

```
.\.venv\Scripts\python.exe -m auction_portal crawl baanknet --limit 20
```

This takes about a minute. It waits three seconds between requests on purpose — we do not hammer anyone's website. Refresh the portal afterwards and the new listings appear.

### See what has been collected

```
.\.venv\Scripts\python.exe -m auction_portal stats
.\.venv\Scripts\python.exe -m auction_portal listings --state Gujarat
```

### Check for duplicates and re-auctions

```
.\.venv\Scripts\python.exe -m auction_portal dedup
```

Reports two things: the same property listed twice, and the same property being re-auctioned later at a lower price. It **reports only** — nothing is merged automatically, because a wrong merge is hard to notice.

## Run it fully automatically

```
.\.venv\Scripts\python.exe -m auction_portal schedule
```

This is how it runs in production: crawls every 6 hours, re-checks imminent auctions hourly, updates statuses every 15 minutes, and checks its own health every 2 hours. Press **Ctrl+C** to stop.

Add `--dry-run` to list the jobs without starting them.

## 6. Deliberate design decisions

Please read this before reporting these as bugs.

**One listing is hidden.** Of 85 collected, 84 are shown. The hidden one is a Mumbai property where the bank uploaded a reserve price of ₹1 — a placeholder. Rather than display a nonsense price, the system withholds it. You can see it with `listings --all`.

**We show fewer listings rather than wrong ones.** Every listing gets a confidence score. Below 0.70 it is stored but not published. This means slightly lower coverage and no wrong prices. That trade was chosen deliberately.

**Some listings say "unscheduled".** Banks list properties before fixing an auction date. That is normal, not missing data.

**Borrower names are never shown.** Not hidden — never stored. Publishing that someone defaulted causes real harm and adds nothing for a buyer.

**Old auctions disappear after 90 days.** Concluded auctions stop being publicly searchable. The data is kept internally; it just stops being indexed.

## 7. If something goes wrong

| Problem                                 | Fix                                                                      |
|-----------------------------------------|--------------------------------------------------------------------------|
| "cannot connect" / database errors      | <code>python scripts\setup_postgres.py --start</code>                    |
| Portal will not start, "address in use" | Another copy is running. Close it, or use <code>serve --port 8001</code> |
| Page loads but shows no listings        | Database is empty. Run <code>crawl baanknet --limit 20</code>            |
| A crawl fails with network errors       | Check your internet. It retries automatically.                           |
| Prices look wrong                       | Please report it — include the listing URL. This matters most.           |

To restart from scratch:

```
.\.venv\Scripts\python.exe scripts\setup_postgres.py --reset
.\.venv\Scripts\python.exe scripts\setup_postgres.py
.\.venv\Scripts\python.exe -m alembic upgrade head
.\.venv\Scripts\python.exe -m auction_portal crawl baanknet --limit 50
```

## 8. Reporting what you find

For anything wrong, please include:

1. **What you did** — the page, the filters, the exact search
2. **What you expected**
3. **What happened** — a screenshot helps
4. **The listing URL** if it concerns specific data

Priority order for us:

| Priority        | Kind of problem                                    |
|-----------------|----------------------------------------------------|
| <b>Critical</b> | A wrong reserve price, EMD or auction date         |
| <b>Critical</b> | Any borrower or personal detail appearing anywhere |
| High            | A listing linking to the wrong original notice     |
| High            | Filters or search returning wrong results          |
| Medium          | Missing listings, layout problems                  |
| Low             | Wording, styling, ordering preferences             |

The two critical rows are the ones that would genuinely damage a user. Everything else can wait.

## 9. Before this can go live publicly

This system is **not yet ready for public use**, for reasons that are nothing to do with the software:

1. **A lawyer must review it.** An Indian technology and IP lawyer needs to check the data collection approach, the disclaimers, the privacy policy and the position under the Digital Personal Data Protection Act.
2. **A real contact page.** Our collector currently identifies itself with a placeholder address. Before crawling at volume, that must point to a live page explaining what the bot does and how to ask it to stop.
3. **A published takedown process**, with a named contact and a committed response time.
4. **Proper hosting** — managed database, backups, HTTPS, a domain.

Estimated running cost once live: **₹1,000 – 6,500 per month**.

Full detail is in [docs/DEPLOYMENT.md](#).

## 10. Quick reference

---

```
# Start the database (needed after every reboot)
.\.venv\Scripts\python.exe scripts\setup_postgres.py --start

# Start the portal          -> http://127.0.0.1:8000
.\.venv\Scripts\python.exe -m auction_portal serve

# Collect more listings
.\.venv\Scripts\python.exe -m auction_portal crawl baanknet --limit 20
.\.venv\Scripts\python.exe -m auction_portal crawl baanknet_vehicle --limit 20

# Inspect
.\.venv\Scripts\python.exe -m auction_portal stats
.\.venv\Scripts\python.exe -m auction_portal health
.\.venv\Scripts\python.exe -m auction_portal listings --state Gujarat
.\.venv\Scripts\python.exe -m auction_portal dedup

# Run everything automatically
.\.venv\Scripts\python.exe -m auction_portal schedule

# Confirm the code is sound (301 automated tests)
.\.venv\Scripts\python.exe -m pytest
```